23CSE455 Design Patterns

Quarkus Digital Wallet Case Study

Pattern Identification, System Evolution, and Cross-Language Benchmarking

Team B9

Date: 2026-10-11 | Repository: 23CSE455_DP_CaseStudy

1. Executive Summary & Baseline Architecture

23CSE455 Design Pattern Case Study --- Team B9

Case Study Scope & Context:

- Domain: Quarkus-based Digital Wallet application handling balances, transfers, and notifications.
- Target: Java 17, Quarkus 3.40.1, REST/Jackson API, in-memory repository.
- Core Objective: Trace design pattern discovery at baseline, refactor across 3 change requirements,
- and compare cross-language implementations across Java, Python, JavaScript, and C++.

Baseline Pattern Inventory (7 Confirmed Collaborations):

* Builder (Creational): Transaction.Builder accum. fields & validates construction
* Facade (Structural): WalletFacade coordinates repo, command & notifications
* Repository (Enterprise): WalletRepository abstracts in-memory wallet persistence
* Observer (Behavioral): NotificationPublisher broadcasts transfer events to Email/Audit
* State (Behavioral): TransactionState handles PendingState -> CompletedState lifecycle
* Command (Behavioral): TransferCommand encapsulates debit/credit business execution
* Value Object (DDD idiom): Money record enforces positive amount and currency invariants

2. Evolution & Refactoring Requirements

23CSE455 Design Pattern Case Study --- Team B9

CH01: Multi-Currency

Requirement:

Support EUR & USD-EUR FX
without hardcoded if-else.

Patterns Introduced:

* Strategy: CurrencyConversionStrategy
* Simple Factory: CurrencyStrategyFactory
* Adapter: VendorFxAdapter
* CDI Injection: @Inject dependencies
* DTO: REST Boundary records

Refactoring:

Builder replaced by fixed record;
WalletFacade evolves to WalletService.

CH02: Transaction Limits

Requirement:

Per-transfer cap (EUR 1,000) &
daily aggregate cap (EUR 1,000).

Patterns Introduced:

* Strategy: TransactionLimitPolicy
* Decorator: DailyLimitDecorator

Refactoring:

Wraps base per-transfer limit with
daily limit check dynamically.
Prevents bloated conditional
logic inside transfer orchestration.

Acceptance: EUR 900 passes;
EUR 1001 fails; EUR 300 fails if
prior daily usage = EUR 800.

CH03: Fraud Screening

Requirement:

Sequential screening checks before
limits and balance updates.

Patterns Introduced:

* Chain of Responsibility:
  - DenyListCheck (blocked accts)
  - VelocityCheck (> EUR 6,000)

Refactoring:

Replaces fraud if-else blocks
with extensible Handler chain.
Returns first rejection early.

Acceptance: Blocked beneficiary
rejected; > 6000 velocity review;
EUR 200 normal transfer passes.

3. Pattern Counts Evolution Summary

23CSE455 Design Pattern Case Study --- Team B9

Source Snapshot	Pattern Types	Total Instances	Status & Notes
Baseline	7	7	Builder, Facade, Repository, Observer, State, Command, Value Object
After CH01	11	14	+Strategy, Simple Factory, Adapter, CDI DI, DTO; -Builder; +2 Repos
After CH02	12	16	+Strategy (Transaction Limit), +Decorator (DailyLimitDecorator)
After CH03	13	17	+Chain of Responsibility (FraudCheckChain: DenyList, Velocity)

Design Insights:

* Pattern density increased from 7 to 17 instances, isolating all key variation points.
* Physical LOC increased from 92 to 180 lines, accommodating REST endpoints and safety checks.
* More patterns alone do not prove superior design; the tangible benefit is decoupling rate lookup, limits, and fraud rules from the transfer orchestration pipeline.

4. Cross-Language Implementation & Performance

23CSE455 Design Pattern Case Study --- Team B9

Language	Strategy Mechanism	Observer Mechanism	LOC	Max CC	Result	Median ms
Java	Interface + Factory	Lambdas via Interface	23	2	not_run*	-
Python	Duck-typed Classes	Callable Function List	30	2	4/4 passed	93.29 ms
JavaScript	Classes (Minor Units)	Function Array Broadcast	15	2	4/4 passed	63.37 ms
C++	Abstract Base + unique_ptr	std::function Lambdas	27	2	not_run*	-

*** Measurement & Execution Notes:**

- Timing protocol: 1 warmup run + 5 measured executions on same machine; median reported.
- Measured times are whole-process startup + execution durations, not isolated throughput loops.
- Java & C++ are recorded strictly as not_run because JDK & C++ compilers were not in the local image; Assignment policy strictly forbids estimating missing benchmark values.

5. Verification, Limitations & Deliverables

23CSE455 Design Pattern Case Study --- Team B9

AI (LLM) Verification Summary:

- All pattern candidate claims from LLM sessions were verified against actual source code AST & call hierarchy.
- LLM prompts, raw outputs, and human evaluations preserved under Part1/llm, Part2/llm, and Part3/llm.

Key Production Limitations & Next Steps:

- In-Memory Persistence: Needs relational DB (PostgreSQL / Panache) with ACID transaction isolation.
- Distributed Idempotency: Duplicate payment prevention requires Redis/distributed lease lock.
- Live FX & Security: Dynamic quote provider with audit trails and PCI DSS compliance needed.

Repository Package Structure:

B9/ folder ready for submission: README.md, Report.pdf, Report.tex, data/, Part1/, Part2/, Part3/.